



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS344 - Computer Graphics

Copyright ©2026 by FJ Redelinghuys. All rights reserved.

Practical 2 Specification: 2D Rendering

Release Date: 16-02-2026 at 06:00

Start By Date: 28-02-2026

Due Date: 23-03-2026 at 10:00

Total Marks: 110

Contents

1	General Instructions	3
2	Overview	4
3	Your Task:	4
3.1	Layout Requirements	4
3.2	Shape Requirements	5
3.3	Colour Requirements	5
3.4	Selection Requirements	6
3.5	Transformation Requirements	6
3.6	Wireframe	8
4	Marking rubric	9
5	Bonus marks	9
6	Implementation Details	12
7	Submission	12
8	Demo Instructions	13

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departamental-guide/>.
- Unless otherwise stated, the usage of additional libraries outside of those indicated in the assignment, will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++11**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.
- No pre-build objects and textures may be used. All objects and textures that you need to use must be created by yourself.
- You must use OpenGL version 3.3 for this practical.

2 Overview

For this practical, you will need to render a scene populated with simple 2D objects and apply a set of transformations to the objects.

3 Your Task:

For this practical, you will need to render a simple 2D layout for a single hole in a miniature golf course. This will help you design and lay out the miniature golf course for the homework assignment.

Hint: It is recommended that you read Section 3.1.2 of the homework assignment while designing your layout. Each type of obstacle/decor should be represented with a unique colour and shape combination. An illustrative example of a layout is given in Figure 1.

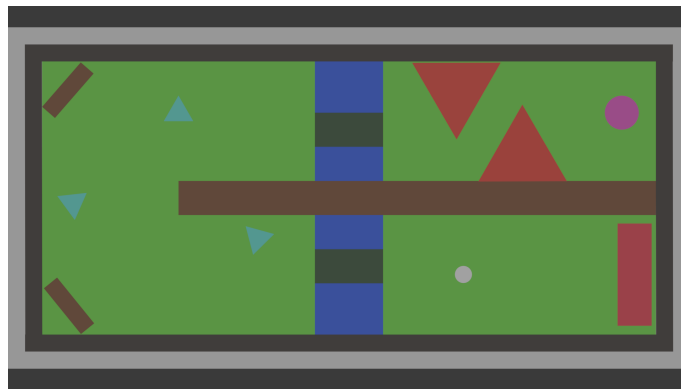


Figure 1: Layout Example

In the sections below, the different requirements are explained. You are allowed to use any colour/shape combination to represent different types of objects/decorations in the floor plan. You may orient the floor plan in any direction you like, as long as it is a realistic representation of a 2D layout.

3.1 Layout Requirements

The layout should have the following example visual components¹:

- A shape indicating the concrete floor of the course (represented in grey). Note that the floor may not occupy the entire screen. There should be a border between the floor and the edges of the window.
- Starting point for the course (represented by a maroon rectangle).
- River/stream running through the course (represented by a blue rectangle).
- At least 3 different types of obstacles (one type is represented by the light brown rectangles).
- A circle representing the golf hole (represented by the pink circle).

¹All shapes refer to examples in Figure 1

- A circle representing the golf ball (represented by the white circle).
- A set of rectangles (indicated in dark brown) that creates a border between the grass (indicated in green) and the floor.

Your layout can contain more details if you wish to add them.

3.2 Shape Requirements

Your layout is required to contain at least two of the following distinct² shape types:

- Rectangle/Square
- Triangle
- Circle
 - One of the circles should consist of at least 50 vertices
 - One of the circles should consist of between 6 and 10 vertices

Note that you can use as many shapes as you like to build your layout, as long as it complies with the above requirements.

3.3 Colour Requirements

Your layout needs to fulfil the following requirements in terms of colour.

- At least 3 distinct colours to represent different types of objects.
- A distinctive colour for the starting point.
- A distinctive colour for the golf hole.
- The water in your layout has to be a distinctive blue.
- The golf ball has to be a distinctive white.
- A distinctive colour for the grass.
- The concrete floor has to be a distinctive grey.
- The background colour also has to be distinctive. *Hint: there is an OpenGL function that can assist with this.*

In conclusion, your layout needs to contain at least 9 different, distinctive, solid colours.

²Distinct implies that the shape as a whole is counted and not the internal polygons used to create the shape.

3.4 Selection Requirements

In order to perform the transformations or animations listed in the following section, you need to be able to select an object to apply the transformation to.

- When the 1 key is pressed, the golf ball needs to be selected.
- When the 2 key is pressed, an obstacle in the golf course needs to be selected.
- When the 3 key is pressed, a different type of obstacle needs to be selected.
- When the 4 key is pressed, the golf hole needs to be selected.
- When the 0 key is pressed, the selected object, if any, needs to be deselected.

The selected object should be highlighted by having the colour change to a pastel version of the original colour. Note that each key can be bound to a single object of the required type.

3.5 Transformation Requirements

In this section, the transformation or animation requirements for your rendering are described. Note, keep the animation rates low such that it allows you to press the key multiple times before going off screen.

- When the W key is pressed, the selected object needs to move from its current position to the top of the screen.
- When the S key is pressed, the selected object needs to move from its current position to the bottom of the screen.
- When the A key is pressed, the selected object needs to move to the left of the screen.
- When the D key is pressed, the selected object needs to move to the right of the screen.
- When the + key is pressed, the selected object needs to be scaled up, from the centre of the selected object.
- When the - key is pressed, the selected object needs to be scaled down, from the centre of the selected object.
- When the E key is pressed, the selected object needs to be rotated clockwise around the centre of the selected object.
- When the Q key is pressed, the selected object needs to be rotated counter-clockwise around the centre of the selected object.

Below are examples of the result after the key is pressed multiple times, assuming the selected object is at the original rendering position.

Note that when a key is pressed, the selected object should **not** return the original position. Your program should be able to apply the transformation in any arbitrary order with the selected object at any position on the screen.

The yellow object represents the selected object.

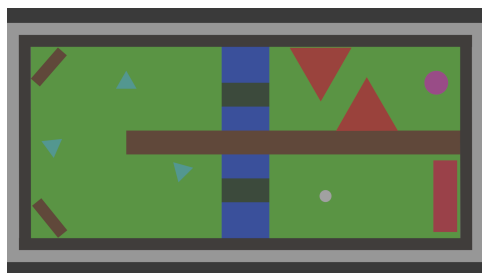


Figure 2: Original

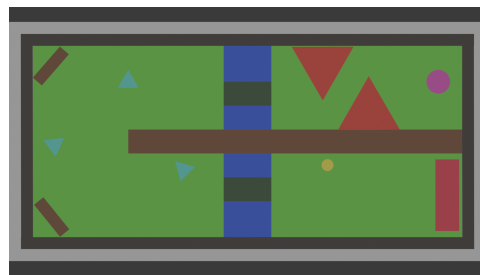
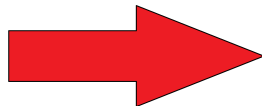


Figure 3: After a series of W key presses

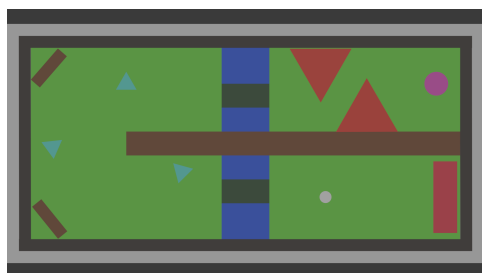


Figure 4: Original

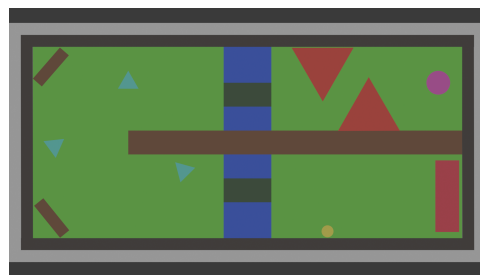
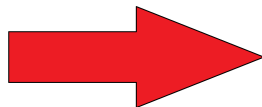


Figure 5: After a series of S key presses

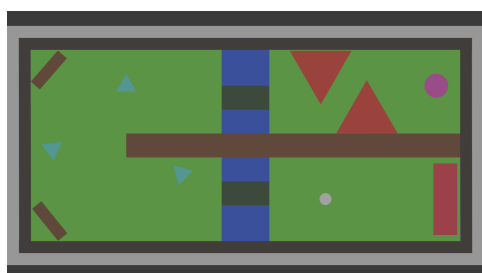


Figure 6: Original

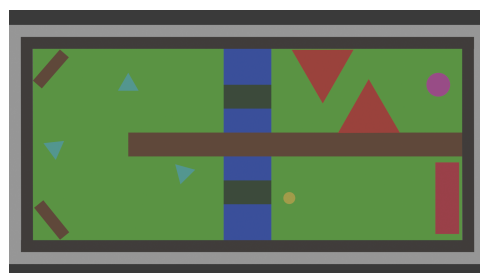
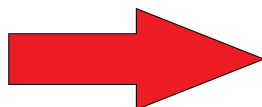


Figure 7: After a series of A key presses

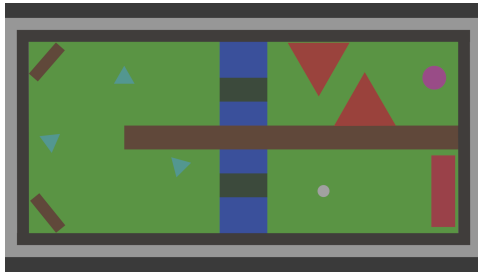


Figure 8: Original

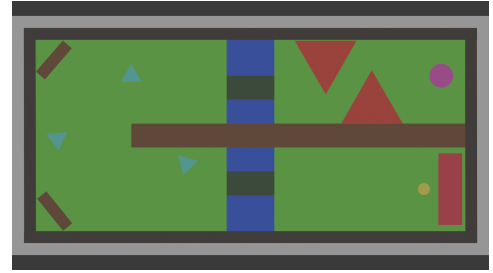
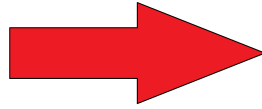


Figure 9: After a series of D key presses

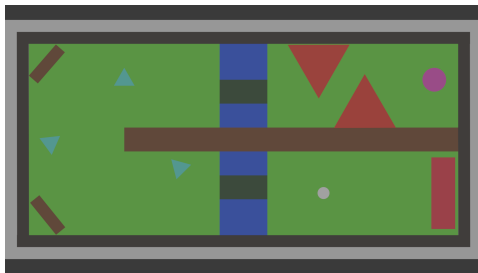


Figure 10: Original

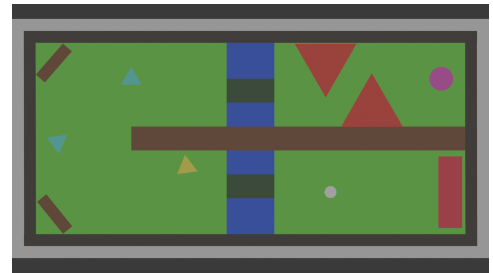
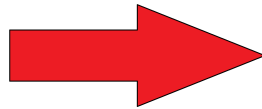


Figure 11: After a series of Q key presses

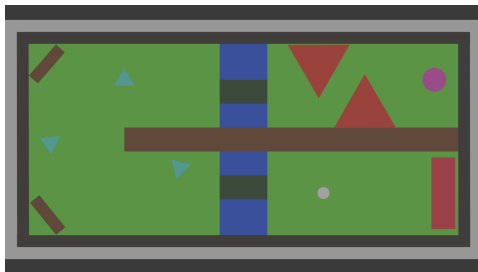


Figure 12: Original

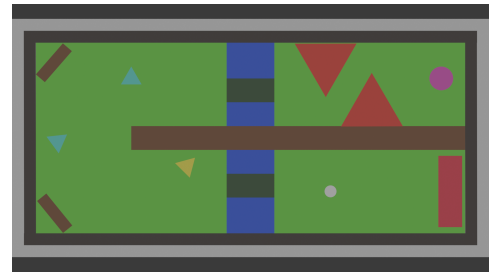
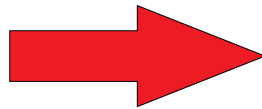


Figure 13: After a series of E key presses

3.6 Wireframe

You are also required to implement a wireframe for your layout. The wireframe should maintain the colour scheme of your floor plan, such that the colours of the different shapes can be identified. The wireframe should also conform to all the transformations described earlier.

Your program should toggle between the wireframe and normal floor plan when the **Enter** key is pressed. *Hint: you may need to implement a time delay between key presses for the wireframe toggling, such that the expected behaviour is achieved.*

Please note that you **must** use the `GL_LINES` to implement your wireframe. Using the `glPolygonMode` function will result in the forfeiture of your wireframe marks.

Please see the figure below for an example of a wireframe rendering of the floor plan.

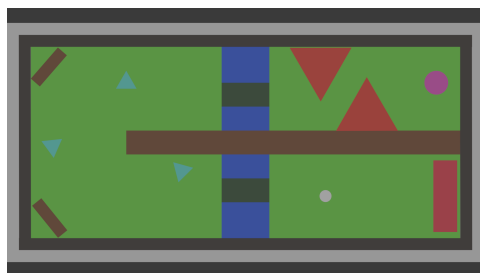


Figure 14: Original

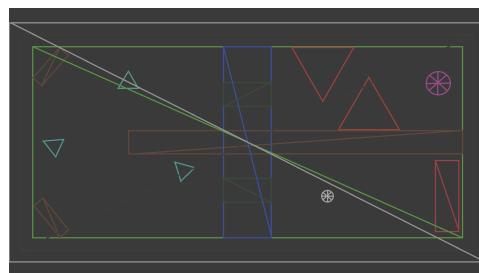
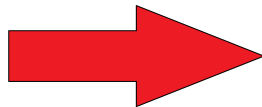


Figure 15: Wireframe

4 Marking rubric

The following rubric will be used to mark your submitted assignment. Note, you will be demoing the practical during the practical sessions on your own computer or a lab computer. Please see Table 1 for the rubric. Note: 1 mark will be subtracted for each transformation assessment criterion if the render is moved back to the centre before a new transformation is applied.

5 Bonus marks

There are up to 10 marks available for bonus marks. The following will only be considered for bonus marks, and are worth 2 marks for each extra that you render or implement (max 10 marks).

- Allowing the selection of more than one object to apply a transformation to.
- Allowing objects to be selected by clicking on them.
- Allowing bounds checking so objects cannot be moved off-screen.
- Allowing bounds checking on obstacles such that the golf ball can't move through them.
- Allowing the golf ball to bounce off the obstacles.
- Enabling the dynamic addition or removal of objects on the screen.
- Adding extra details to objects, such as patterns. Note: you may **not** use in texture mapping for this.
- Being able to save the layout to a text file.
- Being able to load the layout from a text file.
- Being able to save a screenshot of the floor plan. Note this has to be done by the program using OpenGL.

Assessment Criteria	0	1	2	3	4
<u>Layout Requirements [30 marks]</u>					
Shape indicating concrete floor	No shape		Shape occupies entire window		Correct concrete floor shape
Starting point of course	No starting point			Starting point	
River/stream	No river/stream			River/stream is present	
Three different types of obstacles	No obstacles types	Only one type of obstacle object	Only two types of obstacle objects		Three or more obstacle object types
Golf Hole	No golf hole		Golf hole is wrong shape		Golf hole is the correct shape
Golf ball	No golf ball		Golf ball is wrong shape		Golf ball is the correct shape
Golf ball	No golf ball		Golf ball is wrong shape		Golf ball is the correct shape
Barrier	No barrier		Partially complete barrier		Complete barrier
<u>Shape Requirement [12 marks]</u>					
Two distinct rectangles or squares	No rectangles or squares		Only one distinct rectangle or square		Two or more distinct rectangles or squares
Two distinct triangles	No triangles		Only one distinct triangle		Two or more distinct triangles
Two distinct circles	No circles		Only a low polygon circle	Only a high polygon circle	A high and low polygon circle
<u>Colour Requirements [18 marks]</u>					
Three distinct colours for obstacle objects	No colour	All object types have the same colour		Two distinct colours	Three distinct colours
Distinct starting point colour	No distinct colour		Distinct colour		
Distinct golf hole colour	No distinct colour		Distinct colour		
Distinct water blue colour	No distinct colour		Distinct colour		
Distinct white golf ball colour	No distinct colour		Distinct colour		
Distinct grass colour	No grass colour		Distinct grass colour		
Distinct grey concrete colour	No distinct colour		Distinct colour		
Distinct background colour	No background colour	Background colour not distinct	Background colour distinct		

Assessment Criteria	0	1	2	3	4
<u>Selection Requirements [18 marks]</u>					
Selecting golf ball	Not selectable		Selectable with incorrect key	Selectable with correct key but no colour change	Selectable with correct key press
Selecting obstacle type 1	Not selectable		Selectable with incorrect key	Selectable with correct key but no colour change	Selectable with correct key press
Selecting obstacle type 2	Not selectable			Selectable with correct key but no colour change	Selectable with correct key press
Selecting golf hole	Not selectable		Selectable with incorrect key	Selectable with correct key but no colour change	Selectable with correct key press
Unselecting object type	Not implemented	Implemented but no colour change	Correctly implemented		
<u>Transformation Requirements [20 marks]</u>					
Horizontal translations	The selected shape is not able to move		The selected shape is able to move only in one direction		The selected shape is able to move in both directions
Vertical translations	The selected shape is not able to move		The selected shape is able to move only in one direction		The selected shape is able to move in both directions
Scaling	The selected shape is not able to scale		The selected shape is able to scale only in one direction		The selected shape is able to scale in both directions
Rotation	The selected shape is not able to rotate		The selected shape is able to rotate only in one direction		The selected shape is able to rotate in both directions
Transformations applied to all selected shapes	No shape is able to transform	Only one of the selectable shapes is able to perform all transforms	Only two of the selectable shapes is able to perform all transforms	Only three of the selectable shapes is able to perform all transforms	Four or more of the selectable shapes is able to perform all transforms
<u>Wireframe [12 marks]</u>					
Render	No wireframe		Partial wireframe		Correct wireframe
Colour	No wireframe		Partially coloured		Correctly coloured
Transformations	No wireframe		Partial transformations		Correct transformations

Table 1: Marking Rubric

6 Implementation Details

- You need to use OpenGL version 3.3 for this practical.
- You may **not** use any of the built-in mathematical libraries within the `glm` package. This included matrix arithmetic. *Hint: You may use your practical 1 in this practical.*
- You may **not** use any of the built-in OpenGL functions to generate the shapes for you. You need to create each shape from first principles.
- You may **not** use any of the built-in OpenGL functions to perform the transformations of the shapes. You need to transform each shape from first principles, either explicitly or by using the matrix arithmetic techniques discussed in class.
- You may only use the following C++ and OpenGL libraries:
 - `stdio.h`
 - `stdlib.h`
 - `iostream`
 - `iomanip`
 - `initializer lists`
 - `cmath`
 - `sstream`
 - `GL/glew.h`
 - `GLFW/glfw3.h`
 - `glm/glm.hpp`

You may also use the `shader.hpp` and `glad.c` files that assist with compiling and linking of shaders. Your code should be able to be compiled without the assistance of an IDE, i.e. the project needs to be able to be compiled from the terminal.

- All your helper classes and files need to be in the same directory as the `main.cpp`.
- Ensure that the title of the window of your program is your correct student number.

7 Submission

You are required to submit on ClickUp under the appropriate submission link. In the archive that you submit, include a makefile and compiling instructions such that the program can be compiled and executed by the markers if needed. *Failure to upload to ClickUP will result in you forfeiting all marks for this practical.* No exceptions will be made on this matter.

8 Demo Instructions

1. You will first be required to download your submission from ClickUP.
2. You will then demo your practical to the tutor.
3. In the presence of the tutor, you will be required to upload the archive you downloaded from ClickUP to FitchFork. *Failure to upload to FitchFork will result in you forfeiting all marks for this practical.* No exceptions will be made on this matter.